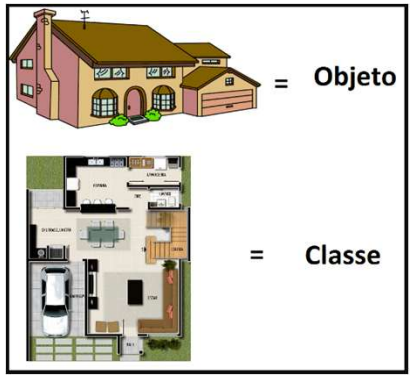
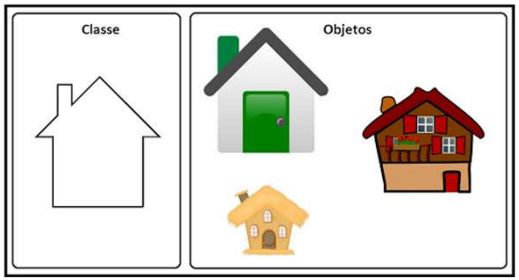
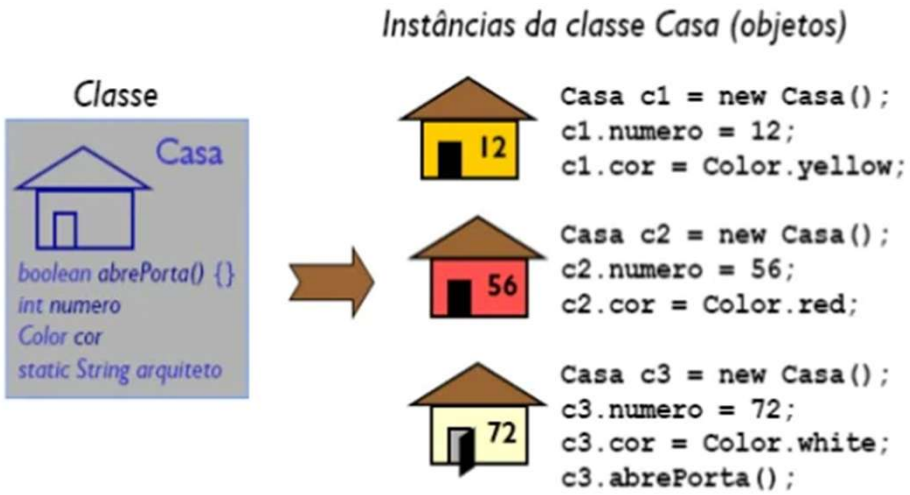
04 de 20 - T2 10/03/2026 - T1 12/03/2026

RECAPITULANDO..

Projeto Classe







- O que todo Livro **tem** e é importante define seus **ATRIBUTOS**, **propriedades**, **características** ou também chamado de **estado**.
- O que todo Livro **faz** e é importante define seus **COMPORTAMENTOS**.

- Quando definimos os **atributos** e **comportamentos**, criamos o **projeto** de nosso Livro, e este projeto é o que chamamos de **classe**.
  - A palavra classe vem da taxonomia da biologia.
  - Todos os seres vivos de uma mesma classe biológica têm uma série de atributos e comportamentos em comum, mas não são iguais, podem variar nos valores desses atributos e como realizam esses comportamentos

- Com isso, temos o **projeto** de um Livro da nossa biblioteca.
- Podemos pegar esse projeto e cadastrar um livro? Não!
- Antes, precisamos pegar o projeto, e com base nele, **construir um livro**, para poder acessar o que ele tem, e pedir a ele que faça algo.
- Há isto damos o nome de **objetos**, e executamos isto através da realização da sua **instância** utilizando o operador **new**.

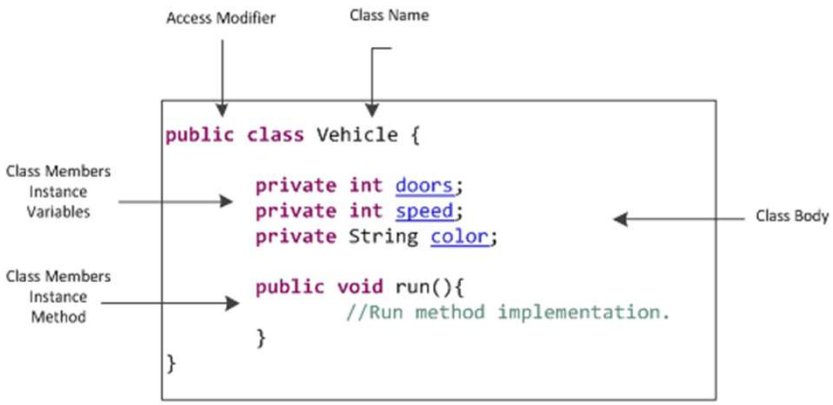
```
namespace Biblioteca;

public class Livro
{
    public string Titulo;
    public string SubTitulo;
    public string Escritor;
    public string Editora;
    public string Genero;
    public int AnoPublicacao;
    public string TipoDaCapa;
    public int NumeroDePaginas;
}
```

```
namespace Biblioteca;

public class Leitor
{
    public string Nome;
    public byte Idade;
    public List<Livro> LivrosLeitor = new List<Livro>();

    public void AdiconarLivro(Livro livro)
    {
        this.LivrosLeitor.Add(livro);
    }

    public void AdiconarLivro(Livro livro)
    {
        this.LivrosLeitor.Add(livro);
    }

    public void DoarLivro(Livro livroDoado, Leitor leitorDestino){
       this.LivrosLeitor.Remove(livroDoado);
       leitorDestino.LivrosLeitor.Add(livroDoado);
    }
}
```

```
namespace Biblioteca;

internal class Program
{
    private static void Main(string[] args)
    {

        Console.WriteLine("Console também é uma classe...");
        Console.WriteLine("WriteLine também é um comportamento...");

    }
}
```

- As classes devem ser criadas em um arquivo isolado, com a extensão **.cs**.
  - É uma boa prática o nome do arquivo possuir o mesmo nome da classe.

- Conforme podemos ver no exemplo ao lado, é possível ter várias classes no mesmo arquivo, mas isso **não é considerado uma boa prática**, e **deve ser evitado**!



BibliotecaPessoal.cs 7

BibliotecaPessoal.cs > ...

```
1   namespace IntroducaoPOO;
2
    2 references
3   public class Leitor
4   {
        2 references
5       public string Nome;
        2 references
6       public byte Idade;
7
        2 references
8       public List<Livro> LivrosLeitor = new List<Livro>();
9
10
        3 references
11      public void AdiconarLivro(Livro livro)
12      {
13          this.LivrosLeitor.Add(livro);
14      }
15  }
16
    12 references
17  public class Livro
18  {
        4 references
19      public string Titulo;
        2 references
20      public string SubTitulo;
        4 references
21      public string Escritor;
        0 references
22      public string Editora;
        0 references
23      public string Genero;
        0 references
24      public byte AnoPublicacao;
        0 references
25      public string TipoDaCapa;
        0 references
26      public int NumeroDePaginas;
27  }
28
```

```
namespace Biblioteca;

internal class Program
{
    private static void Main(string[] args)
    {
        // declaração de variável
        Livro l1;

        // construção/instanciação do objeto
        l1 = new Livro();

        // acesso através da variável de instância
        l1.Titulo = "O contador de Histórias";
        l1.SubTitulo = "Memórias de vida e música";
        l1.Escritor = "Dave Grohl";

        Livro l3 = new Livro();
        l3.Titulo = "Memórias de um legionário";
        l3.Escritor = "Dado Villa-Lobos";


        Leitor leitor01 = new Leitor();
        leitor01.Nome = "Luciano Coelho";
        leitor01.Idade = 46;

        leitor01.AdiconarLivro(l1);
        leitor01.AdiconarLivro(l3);


        Console.WriteLine($"{leitor01.Nome}, {leitor01.Idade} anos.");
        Console.WriteLine($"Livros: ");
        foreach (var livro in leitor01.LivrosLeitor)
        {
            Console.WriteLine($"{livro.Titulo}, {livro.Escritor}");
        }

    }
}
```

- O **objeto** nada mais é do que a **materialização** de uma classe.
- Para criar um **objeto** de uma classe basta **instanciá-la** através do uso da palavra reservada **new**.
- A criação de um objeto permite **armazenar estados desse objeto**, que nada mais são do que atribuição de valores aos atributos.

- As classes fazem parte da nossa rotina diária de desenvolvimento, por exemplo, mesmo antes de saber o que era uma classe, já estávamos utilizando **Program**, **Convert** e **Console** que também são classes.

- Se pegarmos a classe **Console** como exemplo, utilizamos os **atributos**/propriedades **BackgroundColor** e **ForegroundColor,** que armazenam o estado das cores de fundo e da letra que aparecem no console do programa.
- Ainda na classe Console, utilizamos os **comportamentos** de leitura e escrita no console, invocando os métodos **ReadLine()** e **WriteLine()**.

- Da classe **Convert**, entre outros comportamentos, utilizamos o **ToInt32()**.

- **Convert** e **Console** tem em comum o **namespace**, ambas pertentem ao **System**.

RETORNO

Vamos ao trabalho...

Construtores de classes

- Sempre que utilizamos a palavra reservada **new**, estamos executado o **construtor da classe**, e ele é responsável por executar a **lógica de criação do objeto**.
- Toda classe possui um construtor, mesmo que implícito.
  - Quando não declaramos nenhum construtor para a classe, por padrão ele é vazio, e tem o mesmo efeito que o declarado no código ao lado.
- O construtor da classe é um **bloco** declarado com o **mesmo nome que a classe**.

```
namespace Biblioteca;

class Livro
{
    public string Titulo;
    public string SubTitulo;
    public string Escritor;
    public string Editora;
    public string Genero;
    public int AnoPublicacao;
    public string TipoDaCapa;
    public int NumeroDePaginas;


    public Livro()
    {

    }

}
```

```
namespace Biblioteca;

class Livro
{
    public string Titulo;
    public string SubTitulo;
    public string Escritor;
    public string Editora;
    public string Genero;
    public int AnoPublicacao;
    public string TipoDaCapa;
    public int NumeroDePaginas;


    public Livro()
    {
        Titulo = "";
        SubTitulo = "";
        Escritor = "";
        Editora = "";
        Genero = "";
        AnoPublicacao = 0;
        TipoDaCapa = "";
        NumeroDePaginas = 0;
    }

}
```

- É comum utilizarmos o construtor da classe para **inicializar** os atributos do objeto no momento da sua criação.
- Será utilizado padrões que atendam a **lógica de criação do objeto**, sendo comum o uso de valores **default** dos tipos de dados utilizados.
- Com isso garantimos a inicialização dos atributos do objeto evitando erros em tempo de execução.
- No exemplo ao lado, temos a criação do construtor **default**, ou **padrão**. Ele não possui lista de parâmetros.

```
Livro l5 = new Livro();

Console.WriteLine($"{l5.Titulo} , {l5.Escritor}");
```

Sobrecarga de construtores

- No passo anterior criamos um construtor da classe **Livro()** que somente inicializa todos os atributos da classe com seus valores **default**.

- Porem teremos muitos casos onde já queremos no momento da criação do objeto passar argumentos com os valores a serem atribuídos ao objeto, precisando assim, definir parâmetros no construtor.

- **Parâmetros no construtor** são utilizados para casos onde é desejável passar informações na criação do objeto, como por exemplo, quando criamos um **Livro** e já queremos definir um titulo, editora, ano etc.

- Como já possuímos um construtor em nossa classe, para poder criar um novo que atenda essa nova necessidade, precisaremos utilizar um recurso chamado **sobrecarga**, técnica que permite ter **métodos** com o **mesmo nome**, **alterando somente a sua assinatura**.

- Desta forma poderemos criar em nossa classe quantos construtores precisarmos, **alterando somente sua lista de parâmetros**. Assim no momento do **new**, conforme a **lista de argumentos**, o construtor apropriado será executado.

```
namespace Biblioteca2;

public class Livro
{
    public string Titulo;
    public string SubTitulo;
    public string Escritor;
    public string Editora;
    public string Genero;
    public int AnoPublicacao;
    public string TipoDaCapa;
    public int NumeroDePaginas;

    public Livro()
    {
        Titulo = "";
        SubTitulo = "";
        Escritor = "";
        Editora = "";
        Genero = "";
        AnoPublicacao = 0;
        TipoDaCapa = "";
        NumeroDePaginas = 0;
    }

    public Livro(string titulo, string subTitulo, string escritor, string editora, string genero, int anoPublicacao, string tipoDaCapa, int numeroDePaginas)
    {
        Titulo = titulo;
        SubTitulo = subTitulo;
        Escritor = escritor;
        Editora = editora;
        Genero = genero;
        AnoPublicacao = anoPublicacao;
        TipoDaCapa = tipoDaCapa;
        NumeroDePaginas = numeroDePaginas;
    }

}
```

- Reparem que agora possuímos duas formas de construir nosso objeto. Na primeira criamos o objeto com os valores default dos atributos e na segunda já criamos o objeto passando os valores dos atributos via argumentos.

```
1/2 Livro.Livro()
Livro l6 = new Livro()
```

```
Livro l5 = new Livro();
l5.Titulo = "Abc5";
l5.Escritor = "Bolinhas5";

Console.WriteLine($"{l5.Titulo} , {l5.Escritor}");
```

```
2/2 Livro.Livro(string titulo, string subTitulo, string
    escritor, string editora, string genero, int
    anoPublicacao, string tipoDaCapa, int
    numeroDePaginas)
Livro l6 = new Livro()
```

```
Livro l6 = new Livro("Abc6", "Bolinhas6", "Zé", "Edit", "Drama", 2000, "Comum", 350);

Console.WriteLine($"{l6.Titulo} , {l6.Escritor}");
```

- Na maioria das classes usadas no mundo real da programação, os construtores sobrecarregados são a norma, e não a exceção.
- Por exemplo, podemos incluir um terceiro construtor em nossa classe que permita criar um novo objeto passando no momento da criação somente o **titulo** do livro e seu **ano de publicação**.
- Ou seja, criamos construtores conforme nossa necessidade!

```
public Livro(string titulo, int anoPublicacao)
{
    Titulo = titulo;
    SubTitulo = default;
    Escritor = default;
    Editora = default;
    Genero = default;
    AnoPublicacao = anoPublicacao;
    TipoDaCapa = default;
    NumeroDePaginas = default;
}
```

```
Livro l5 = new Livro();
l5.Titulo = "Abc5";
l5.Escritor = "Bolinhas5";
Console.WriteLine($"{l5.Titulo} , {l5.Escritor}");

Livro l6 = new Livro("Abc6", "Bolinhas6", "Zé", "Edit",
"Drama", 2000, "Comum", 350);
Console.WriteLine($"{l6.Titulo} , {l6.Escritor}");


Livro l7 = new Livro("Abc7", 2000);
Console.WriteLine($"{l7.Titulo} , {l7.Escritor}");
```

- Agora que possuímos **construtores**, fica mais fácil imaginar isso virando um programa, onde podemos armazenar nossos leitores e seus respectivos livros.

- Para isso vamos utilizar uma **List** chamada **Leitores**.

```
namespace Biblioteca2;

internal class Program
{
    List<Leitor> Leitores = new List<Leitor>();

    Program()
    {
        // cria leitores
        Leitor leitor1 = new Leitor("Luciano Coelho", 17);
        Leitor leitor2 = new Leitor("Abc Bolinhas", 12);

        // cria livros
        Livro l1 = new Livro("O contador de Histórias", "Memórias de vida e música", "Dave Grohl", "Editora1", "Biografia", 2021, "Dura", 300);

        Livro l2 = new Livro("O guia do Mochileiro das Galáxias", "Não entre em pânico", "Douglas Adams", "Editora2", "Ficção Científica", 1979, "Dura", 200);

        Livro l3 = new Livro("Memórias de um legionário", 2021);
        l3.Escritor = "Dado Villa-Lobos";

        Livro l4 = new Livro();
        l4.Titulo = "O Senhor dos Anéis";
        l4.Escritor = "J.R.R. Tolkien";
        l4.AnoPublicacao = 1954;

        // adiciona livros aos leitores
        leitor1.AdicionarLivro(l1);
        leitor1.AdicionarLivro(l2);
        leitor1.AdicionarLivro(l3);

        leitor2.AdicionarLivro(l4);

        // não temos a necessidade da variável de instancia para poder criar e adicionar um livro, objeto
        leitor2.AdicionarLivro( new Livro("Cdf", 2024) );

        // adiciona leitores a lista de leitores
        Leitores.Add(leitor1);
        Leitores.Add(leitor2);
        Leitores.Add(new Leitor("Joãozinho", 10));
    }

    private static void Main(string[] args)
    {
        _ = new Program();
    }
}
```

- No List **Leitores**, estamos armazenando **referencias** dos objetos que foram armazenados, no caso, referencias de objetos do tipo **Leitor**.

- Nossos objetos do tipo Leitor estão armazenados no List, mas não temos mais acesso as **variáveis de instancia** que utilizamos para realizar o armazenamento, por exemplo, não temos mais acesso ao identificador **leitor1**, ou então, nem mesmo existiu uma variável de instancia para realizar o armazenamento.

- Para podermos editar um objeto, antes precisamos ter uma variável de instancia com referencia para ele, ou seja, se quisermos adicionar os livros do leitor que possui Nome **Cdf**, primeiro precisamos localizar este leitor e referenciar ele em um identificador.

- Veja alguns exemplos...

```
// utilizando a forma tradicional
foreach (var auxLeitor in Leitores)
{
   if (auxLeitor.Nome == "Cdf")
   {
      auxLeitor.AdicionarLivro(new Livro("novo livro", 2024));
      break;
   }
}
```

```
// utilizando LAMBDA e LINQ
var filtradas = Leitores.Where((Leitor l) => { return l.Nome.Equals("Cdf"); });
foreach (Leitor auxL in filtradas)
{
   auxL.AdicionarLivro(new Livro("novo livro dsd2", 2010));
}


// utilizando LAMBDA e LINQ
Leitor c = Leitores.First((Leitor l) => { return l.Nome.Equals("Cdf"); });
c.AdicionarLivro(new Livro("novo livro dsd2", 2010));


// utilizando LAMBDA e LINQ com sintaxe simplificada
var auxFiltradas = from cc in Leitores where cc.Nome.Equals("Cdf") select cc;
foreach (Leitor cc in auxFiltradas)
{
   cc.AdicionarLivro(new Livro("novo livro dsd2", 2010));
}
```

```
// utilizando método Find
Leitor? auxLeitorAddLivro2 = Leitores.Find(x => x.Nome == "Cdf");
if (auxLeitorAddLivro2! != null)
{
   auxLeitorAddLivro2.AdicionarLivro(new Livro("novo livro 2", 2020));
}
```

ATENÇÃO
DESVIO

Material
complementar

Lambda

- As **expressões lambda** ou apenas **Lambda**, são funcionalidades comuns em muitas linguagens, particularmente falando, nas **linguagens Funcionais** (Programação Funcional, LISP, Ruby, JavaScript, F# etc) e o termo Lambda vem do Cálculo Lambda, ou seja, da matemática.
    - "Em ciência da computação, **programação funcional** é um paradigma de programação que trata a computação como uma avaliação de funções matemáticas e que evita estados ou dados mutáveis. Ela enfatiza a aplicação de funções, em contraste da programação imperativa, que enfatiza mudanças no estado do programa.“
- Primeiramente Expressões Lambda são **funções anônimas**, a diferença de uma função normal é que elas não possui **nome** então eles acabam sendo definidas diretamente muitas vezes em uma única linha, e serve para fornecer abstrações para problemas complexos e isso acaba limpando o código deixando o escopo menor.
- Elas estão presentes nas linguagens como Java, C#, Python, Ruby e entre outras.
- As principais vantagens sobre a utilização da expressão lambda:
    - Código mais simples;
    - Simplificar diversas operações em cima de uma coleção de dados;
    - Possui uma sintaxe extremamente simples.
- E as poucas desvantagens:
    - Funções anônimas podem gerar problemas na hora de depurar o código;
    - Muitos argumentos em uma Expressão Lambda gera legibilidade mais difícil;

LINQ

https://learn.microsoft.com/pt-br/dotnet/csharp/linq/get-started/introduction-to-linq-queries

- **LINQ** é um acrônimo para **Language Integrated Query**, ou **Consulta Integrada à Linguagem**.

- Trata-se de um “framework” dentro do .NET destinado a auxiliar os desenvolvedores a escrever expressões de consulta diretamente em C# de maneira agnóstica, sem conhecer necessariamente o que acontece internamente.

LAMBDA e LINQ

- Para passar um bloco de código que pode ser utilizado por um método, o C#/Java introduziu as **funções anônimas** ou **lambdas**.
- As funções anônimas funcionam como métodos estáticos da linguagem com uma declaração simplificada.
- Para declarar uma função anônima que recebe um argumento do tipo **Leitor** utilizamos o seguinte código:

```
(Leitor c) => { // implementação da função anônima }
```

- Dentro do bloco de implementação da função anônima, colocaremos a implementação da condição:

```
(Leitor c) => { return c.Cpf.Equals("2"); }
```

- E agora essa função poderia ser passada para dentro de um método, por exemplo um método chamado **Filtrar**:

```
leitores.Filtrar(    (Leitor c) => { return c.Cpf.Equals("2"); }   );
```

- No C# temos exatamente a implementação dessa ideia, mas o método se chama **Where**, e é integrante de **System.Linq**.
- Então, para buscarmos todas os leitores que têm um cpf igual a "**2**", por exemplo, utilizaríamos o seguinte código:

```
var filtradas = leitores.Where(  (Leitor c) => { return c.Cpf.Equals("2");   });
```

  - Podemos simplificar ainda mais o comando, removendo **(Leitor c)**, pois isso vai ser pego automaticamente de **leitores**.
  - E quando declaramos uma função anônima que tem apenas uma linha que devolve um valor, podemos remover inclusive as chaves e o return da declaração do lambda.

```
var filtradas = leitores.Where( c => c.Cpf.Equals("2")  );
```

- Agora que temos a lista de leitores filtradas, podemos, por exemplo, iterar nessa lista:

```
foreach (Leitor c in filtradas)
{
    MessageBox.Show( c.ToString() );
}
```

- Através do LINQ também podemos utilizar outros métodos, por exemplo:
  - Select, Where, Distinct, ElementAt, ElementAtOrDefault, DefaultIfEmpty, Contains, Single, SingleOrDefault, First, FirstOrDefault, Last, LastOrDefault, All, ToDictionary, Any, OrderBy, ThenBy, Reverse, Sum, Average, Count, Min, Max, Skip, Take, Concat, Union, Range

MELHORANDO AS BUSCAS UTILIZANDO A SINTAXE DE QUERIES

- Vimos que utilizando o **LINQ** podemos fazer filtros e agregações de uma forma fácil em qualquer coleção do C#, porém quando precisamos fazer um filtro complexo, o **lambda** pode ficar com um código complexo. E por isso, a Microsoft decidiu facilitar ainda mais o uso do LINQ.
- Para implementarmos um filtro, em vez de utilizarmos o método **Where**, podemos utilizar uma sintaxe que foi baseada na linguagem de busca em banco de dados, a SQL.

- Para começarmos um filtro utilizando essa nova sintaxe, precisamos começar o filtro com a palavra **from** criando uma variável que será utilizada para navegar na lista:

```
var filtradas = from c in leitores
```

- Agora para colocarmos uma condição nesse filtro, utilizamos a palavra **where** passando qual é a condição a que o compromisso deve obedecer para aparecer como resultado desse filtro:

```
var filtradas = from c in leitores          where c.Cpf.Equals("2")
```

- E por fim, precisamos apenas informar o que será selecionado utilizando o select:

```
var filtradas = from c in leitores          where c.Cpf.Equals("2")          select c;
```

```
var filtradas = from c in leitores where c.Cpf.Equals("2") select c;

foreach (Leitor c in filtradas)
{
     MessageBox.Show(c.ToString());
}
```

- Com esse código, estamos definindo um filtro que devolverá apenas os leitores que têm cpf igual a **2**.
- Quando o compilador da linguagem C# encontra o filtro que definimos, esse código é convertido para uma chamada para o método **Where** que vimos anteriormente.
- Essa sintaxe é apenas um jeito de esconder a complexidade do lambda.

ORDENANDO COLEÇÕES COM LINQ

- Além de fazermos buscas e projeções, podemos também utilizar o LINQ para ordenar coleções de elementos. Para isso precisamos apenas colocar um **orderby** dentro da **query**.

- Por exemplo, para buscarmos todas as contas com saldo maior do que 10000 ordenadas pelo nome do titular, utilizamos o seguinte código:

```
List<Conta> contas = // inicializa a lista de contas

var resultado = from c in contas        where c.Saldo > 10000        orderby c.Titular.Nome        select c;
```

- Com isso temos uma lista de contas ordenadas pelo nome do titular de forma ascendente (alfabética). Assim como podemos fazer a ordenação ascendente, também podemos fazer a ordenação descendente utilizando a palavra descending:

```
List<Conta> contas = // inicializa a lista de contas

var resultado = from c in contas        where c.Saldo > 10000        orderby c.Titular.Nome descending        select c;
```

- Mas e se tivermos dois titulares com exatamente o mesmo nome? Nesse caso, podemos definir um segundo critério para desempatar a ordenação. Cada um dos critérios da ordenação fica separado por vírgula no orderby . No exemplo, para desempatarmos a ordenação utilizando o número da conta, utilizamos o seguinte código:

```
List<Conta> contas = // inicializa a lista de contas

var resultado = from c in contas        where c.Saldo > 10000

orderby c.Titular.Nome descending, c.Numero        select c;
```

ORDENANDO COLEÇÕES COM LINQ

- O segundo critério de ordenação também pode ter opcionalmente a palavra descending :

```
List<Conta> contas = // inicializa a lista de contas
var resultado = from c in contas where c.Saldo > 10000
orderby c.Titular.Nome descending, c.Numero descending select c;
```

- Assim como no caso do filtro, as ordenações do LINQ também são traduzidas para chamadas de método pelo compilador do C#. Quando colocamos um orderby na busca, o compilador chama o método OrderBy (ou OrderByDescending no caso de uma ordenação descendente). A query com o filtro e a ordenação pelo titular fica da seguinte forma:

```
var resultado = contas
.Where(c => c.Saldo > 10000)
.OrderBy(c => c.Titular.Nome);
```

- Quando colocamos uma ordenação secundária, o compilador do C# chama o método ThenBy (ou ThenByDescending no caso de uma ordenação secundária descendente):

```
var resultado = contas
.Where(c => c.Saldo > 10000)
.OrderBy(c => c.Titular.Nome)
.ThenBy(c => c.Numero);
```

RETORNO

Conceito	Descrição	Representação
Objeto	Representa uma parte do mundo real	Classe
Características de um objeto	Representa uma descrição de um objeto	Atributos da classe
Ação/Comportamento do objeto	Representa uma ação/comportamento que um objeto pode fazer	Métodos da classe

A programação orientada a objetos (POO) proporciona uma forma entendível e humana de expressar a realidade para o computador.

Através dessa técnica, podemos ler com mais facilidade os códigos de um programa de computador, facilitando o entendimento e sua criação.

Agora é com Você!

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02
2 parciais de nota – atividade em DUPLA

- Com foco no pilar Abstração, utilizando os conceitos iniciais de **POO** e a exemplificação já trabalhada, crie a primeira versão do nosso programa de Biblioteca!

- Deve possuir uma espécie de menu, que permita ao usuário escolher o que deseja realizar, exemplo:
  - CRUD de leitor (cadastrar, listar, editar, excluir)
  - Incluir os livros do leitor
  - Editar um livro especifico do leitor
  - Remover um livro, por exemplo, que foi perdido
  - Doar um livro para outro leitor
  - Listar todos os leitores e seus respectivos livros
  - Listar um leitor especifico e seus respectivos livros
  - Pesquisar por um livro especifico, e mostrar os dados do leitor

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02
2 parciais de nota – atividade em DUPLA

- Nas classes devem ser utilizados construtores.
- Os dados cadastrados devem ser digitados pelo usuário.
- No **leitor** incluir um atributo para o **Cpf**, garantindo assim um identificador único para o leitor.
  - Ao cadastro um novo leitor, validar se o CPF informado já não esta em uso
  - Ver método **Exists** de **List**
- A busca pelo leitor deve ser realizada através do seu CPF
- O usuário deve poder cadastrar quantos leitores ou livros ele desejar

LABORATÓRIO PRÁTICO
PARCIAL DE NOTA AVALIAÇÃO 02
2 parciais de nota – atividade em DUPLA

- **Data de entrega única**
  - **T2 e T1: 22/03**

- Poste o seguinte:
  - Arquivos **.cs** desenvolvidos
    - No código fonte obrigatoriamente deve ter um comentário com seus nomes completos
  - **Vídeo, breve, gravando a tela do computador, demonstrando o funcionamento de todas as funcionalidades**.
    - Não será considerada a gravação via celular! Utilize algum programa que grave a tela do computador mesmo, por exemplo, OBS.
  - **OBRIGATORIAMENTE no vídeo deve aparecer o cadastro de um leitor com o nome de cada membro da dupla!**
- A entrega de arquivos compactados **NÃO** será considerada.
- Somente um aluno entrega, incluindo um comentário com o nome da dupla.

- A aula do dia 17 e 19 será liberada para que as duplas trabalhem na implementação, estarei no laboratório caso optem por trabalharem na UNIPLAC.

- Aproveitem para se reunirem e trabalharem na implementação da atividade!